

Beyond the Sigmoid Function

So far, we have only used the **sigmoid activation function** ($g(z) = \frac{1}{1+e^{-z}}$). This was a natural starting point because it's the foundation of logistic regression.

However, the sigmoid function is not always the best choice. Its output is always between 0 and 1, which is useful for probabilities (like in an output layer) but can be too restrictive for hidden layers.

- **Motivation:** Consider a hidden neuron trying to learn the "awareness" of a T-shirt. Awareness isn't necessarily a binary (0/1) value or a probability. It could be a number that grows (e.g., from 0 to a very large value representing "gone viral"). A sigmoid function cannot represent this.
-

The ReLU Activation Function

To model values that can be zero or any positive number, a different activation function is more effective: **ReLU (Rectified Linear Unit)**.

- **Definition:** The ReLU function is defined as $g(z) = \max(0, z)$.
 - **How it Works:**
 - If the input z is negative, the output is 0.
 - If the input z is positive, the output is just z .
 - **The Graph:** It's a two-part function: a flat line at 0 for all negative inputs, and a 45-degree line for all positive inputs.
 - **Output Range:** The output of a ReLU neuron is always 0 or a positive number (it is **non-negative**). This makes it an excellent default choice for hidden layers.
-

Commonly Used Activation Functions

Here are the three most common activation functions you will encounter:

1. Sigmoid:

- **Formula:** $g(z) = \frac{1}{1+e^{-z}}$
- **Output Range:** 0 to 1.
- **Use Case:** Good for the **output layer** in a **binary classification** problem (where you need a probability).

2. ReLU (Rectified Linear Unit):

- **Formula:** $g(z) = \max(0, z)$
- **Output Range:** 0 to ∞ (non-negative).

- **Use Case:** The most common and effective default choice for **hidden layers**.

3. Linear:

- **Formula:** $g(z) = z$
- **Output Range:** $-\infty$ to $+\infty$ (any number).
- **Note:** This is often referred to as "no activation function" since the output a is simply equal to the input z .
- **Use Case:** Good for the **output layer** in a **regression** problem (where you need to predict a number, not a probability).

*(A fourth function, **softmax**, will be introduced later for multi-class classification.)*

How to Choose Activation Functions

The choice of activation function is a key part of designing a neural network. The rules for choosing are different for the **output layer** versus the **hidden layers**.

1. Choosing the Output Layer Activation Function

The choice for the output layer is determined by the **type of problem** and the **range of your target variable y** .

Problem Type & y Value	Recommended Activation	Why?
Binary Classification (y is 0 or 1)	Sigmoid	Outputs a probability between 0 and 1, just like logistic regression.
Regression (y can be positive or negative)	Linear ($g(z) = z$)	The output (a) needs to be able to take on any value, positive or negative.
Regression (y is non-negative, ≥ 0)	ReLU ($g(z) = \max(0, z)$)	The output (a) is guaranteed to be 0 or positive, matching the range of y .

This is the most common and recommended approach for setting the output layer.

2. Choosing the Hidden Layer Activation Function

For all hidden layers, the modern default and most common choice is **ReLU**.

- **Recommendation:** Use **ReLU** for all hidden layers.

Why ReLU is Preferred Over Sigmoid (for Hidden Layers)

Historically, sigmoid functions were used in hidden layers, but this practice is now rare. ReLU is preferred for two main reasons:

1. **Faster Computation:** The $\max(0, z)$ operation is computationally simpler and faster than the sigmoid function (which involves an exponent and division).
2. **Faster Training (Gradient Descent):** This is the more important reason.
 - The **sigmoid function** "goes flat" (saturates) in two regions: when z is very large or very small.

- These **flat spots** have a gradient (slope) of almost zero.
 - A zero-gradient **stops gradient descent from learning** or slows it down significantly.
 - The **ReLU function** is only flat in one region (when z is negative). It does not saturate for positive values.
 - This means that, on average, learning (gradient descent) is much faster with ReLU.
-

Implementation in TensorFlow

You specify the activation function as a string in the Dense layer.

- **Example:** A network for binary classification (like the one above) would be built like this:

```
1 model = Sequential([
2     # Hidden Layer 1: Use ReLU
3     Dense(units=25, activation='relu'),
4
5     # Hidden Layer 2: Use ReLU
6     Dense(units=15, activation='relu'),
7
8     # Output Layer: Use Sigmoid
9     Dense(units=1, activation='sigmoid')
10 ])
```

- If this were a regression problem where y could be positive or negative, you would change the last line to `activation='linear'`.

Why Non-Linear Activation Functions are Essential

A neural network **must** use **non-linear activation functions** (like ReLU or sigmoid) in its hidden layers.

If a neural network only uses **linear activation functions** ($g(z) = z$) in all its layers, the entire network becomes computationally equivalent to a simple **linear regression** model. This completely defeats the purpose of having multiple "deep" layers, as the network will be unable to learn complex, non-linear patterns.

Mathematical Proof (The "Collapse") 🧮

Let's examine a simple 2-layer network where every activation function is linear ($g(z) = z$).

1. Layer 1 (Hidden Layer):

- The computation is: $a_1 = g(w_1x + b_1)$
- With a linear activation, this becomes: $a_1 = w_1x + b_1$

2. Layer 2 (Output Layer):

- The computation is: $a_2 = g(w_2a_1 + b_2)$
- With a linear activation, this becomes: $a_2 = w_2a_1 + b_2$

3. Substitute a_1 into a_2 :

- $a_2 = w_2(w_1x + b_1) + b_2$
- $a_2 = (w_2w_1)x + (w_2b_1 + b_2)$

4. Result:

- We can define a new, single $W = w_2w_1$ and a new, single $B = w_2b_1 + b_2$.
- The final output is: $a_2 = Wx + B$

This final equation is just the formula for simple linear regression. No matter how many "deep" layers you add, if they are all linear, they "collapse" into a single linear function.

Key Principle: A linear function of another linear function is, itself, just a linear function.

What About Mixed Functions?

This "collapse" still happens even if the output layer is non-linear (like sigmoid).

- If all **hidden layers are linear** ($g(z) = z$) but the **output layer is sigmoid**:
 - The network collapses and becomes equivalent to a simple **logistic regression** model.
 - You gain no advantage from the "deep" architecture.
-

Conclusion & Rule of Thumb 📜

- **Hidden Layers:** You **must** use a non-linear activation function.
 - **Recommendation:** The default and most common choice is **ReLU**.
- **Output Layer:** You *can* use a linear activation function, but **only** if you are solving a regression problem (as discussed in the previous video).